HACKATHON BUILD · TECHNICAL REPORT

WhiteBoardAI: an AI tutor that talks and draws

Press a hotkey, ask out loud, and a whiteboard next to your screen explains the problem step by step, drawing while it speaks.

`Electron 33` `Node 20 / Python 3.12` `Groq + Gemini` `Piper TTS` `89 tests`

---

1. Executive summary
2. Architecture and tech stack
3. Folder and file structure
4. Core functionality and data flow
5. Key logic highlights
6. Setup and execution

01 Executive summary

WhiteBoardAI is a desktop overlay for students who get stuck while watching a video or reading a page. They press **Ctrl/Cmd+Shift+Space** and say what confuses them. The app takes a screenshot, transcribes the question, and asks a language model for a *script*: a list of short steps, each with a sentence to say and drawing actions to perform. A whiteboard window then speaks each sentence with a local voice while drawing the matching shapes, labels, formulas, graphs or diagrams.

The student can interrupt at any point ("wait, why is that squared?"). The follow-up goes to the model together with the current board and the conversation so far, and the answer continues on the same drawing instead of starting over.

The core value is the format. A chatbot gives a wall of text; this gives the kind of explanation a good teacher gives at a board, paced in small steps, picture before formula, with a check question at the end.

MEASURED ON THE DEV LAPTOP (THINKPAD T490)	TYPICAL
Student stops talking → first step starts (streamed, qwen)	1.0–1.7 s
Follow-up question → first step	0.7–2.0 s
Speech to text (Groq Whisper)	0.3–0.7 s
Making one spoken line (Piper, CPU)	0.3–0.9 s
Fallback model when qwen is rate-limited (gpt-oss)	3–5 s

02 Architecture and tech stack

Two halves meet at one contract. The **shell** (Electron main process plus a preload script) owns everything up to the script: hotkey, screenshot, microphone, speech to text, model calls, layout clean-up and the voice. The **board** (a plain web page loaded into the overlay) owns everything after the script: drawing, animation, layout of notes and the timing that keeps drawing in step with speech. They talk through `shared/contract.d.ts` and `shared/script.schema.json`.

Shell (`shell/`)

- **Electron ^33.0.0** (33.4.11 installed): overlay window, global shortcut, `desktopCapturer` for the screenshot. The only dev dependency in `shell/package.json`.
- **ajv ^8.20.0**: the only runtime npm dependency. Validates every model answer, and every streamed step, against the JSON schema.
- **Node built-ins only** for networking: `fetch`, `FormData`, `AbortSignal.any/timeout`, server-sent-event parsing written by hand. No SDKs.
- **Piper TTS 1.8.0** (`piper-tts` from PyPI) in a private Python venv, voice `en_US-lessac-medium`, run as one long-lived worker process.
- **Models over HTTP**: Groq `qwen/qwen3.8-27b` (the only one that sees screenshots), Groq `openai/gpt-oss-120b`, Gemini `gemini-3.1-flash-lite` and `gemini-3.5-flash-lite` as the last fallback, and Groq `whisper-large-v3-turbo` for speech to text.

Board (`board/`)

- Vanilla JavaScript, no build step. Opens straight from `file://`.

- Vendored libraries: **rough.js 4.6.6** (hand-drawn strokes), **KaTeX 0.19.0** (formulas), **JSXGraph 1.14.0** (function graphs, used headless), **Mermaid 12.2.0** (flowcharts).
- Tests run on `node --test` against a fake DOM (`board/tests/test-board.cjs`).

Storage

There is no database. Keys live in a git-ignored `.env` at the repo root. The last screenshot and the last answer's script are written to `shell/tools/out/` for debugging. Board history is saved to `board-history.json` in the app's data folder (the last 50 boards), so it survives closing the overlay and restarting.

03 Folder and file structure

```
shared/                     the contract both halves agree on
  contract.d.ts             Script, Step, Action types; speak() and playStep() signatures
  script.schema.json        JSON schema the shell validates model output against
  example-pythagoras.json   hand-written sample script

shell/                      Electron app: hotkey to script, plus the voice
  main.js                   window, hotkey, screenshot, IPC wiring, speech cache
  preload.js                speak()/stopSpeech(), mic + silence detection, control bar
  tutor.js                  conversation state machine, streaming step queue
  env.js                    loads the repo-root .env
  llm/callModel.js          the ONE place that talks to models: chain, streaming, retries
  llm/stream.js             pulls complete steps out of a half-written JSON answer
  llm/tidy.js               deterministic layout clean-up of each step
  llm/space.js              occupied boxes, free space, spot finding on the 800×500 board
  stt/transcribe.js         Groq Whisper call
  tts/piper.js              Node side of the Piper worker
  tts/piper_worker.py       loads the voice once, turns lines of text into WAV
  prompts/tutor.md          system prompt: teaching rules, output format, layout rules
  tools/                    llm-test, tts-test, stt-test, board-snapshot, followup-test
  tests/                    41 tests: tutor state machine, model chain, tidy

board/                      the whiteboard page
  index.html                page + test toolbar (hidden inside the app) + history panel
  board.js                  playStep/playScript, drawing, note placement, history
  layout.js                 pure placement logic for notes on the 800×500 board
  cs-helpers.js             array, linked list, binary tree, call stack layouts
  graph-helpers.js          function plots via headless JSXGraph
  flow-helpers.js           Mermaid flowcharts rendered to SVG
  expr.js                   safe formula reader for plots (no eval)
  tests/                    48 tests
```

04 Core functionality and data flow

One question, from keypress to last word:

1. **Hotkey and screenshot.** `main.js` catches Ctrl/Cmd+Shift+Space, grabs the primary screen *before* showing the overlay (so it never captures itself), shrinks it to a 1280-wide JPEG and docks a board-shaped window top-right.
   `main.js · onHotkey, captureScreen`

2. **Listening.** The preload measures room noise for 300 ms, beeps, then records with `MediaRecorder`. Speech means about 150 ms of sound above a clamped threshold; 1.2 s of quiet ends the recording. No speech at all is reported as "heard nothing" rather than sent to Whisper, which invents words like "Thank you." on silence.
   `preload.js · listen`

3. **Speech to text.** The WebM audio goes to Groq Whisper with a vocabulary hint (Pythagoras, hypotenuse, mitochondria...). Each recording carries an id, so a late transcript from a closed session is dropped.
   `stt/transcribe.js · main.js transcribe handler`

4. **Asking the model.** `tutor.js` plays a short filler line ("Okay, let me sketch this out."), then calls `callModel` with the question, the screenshot and, for follow-ups, the board state and the conversation so far. Groq models stream; Gemini answers whole as the last resort.
   `tutor.js · heard → llm/callModel.js`

5. **Steps out of the stream.** `stream.js` watches the JSON as it arrives and hands over each step the moment its braces close. Each step is checked against the schema, then cleaned by `tidy.js`, then queued.
   `llm/stream.js · llm/tidy.js createTidier`

6. **Voice made ahead.** As soon as a step lands, its sentence goes to the Piper worker, so by the time the board reaches it the audio is ready.
   `main.js prepareSpeech · tts/piper.js`

7. **Drawing and speaking.** The tutor calls `window.board.playStep(step, window.shell.speak)` one step at a time. The board starts speech and drawing together and resolves when both finish; between steps the tutor checks whether the answer was interrupted.
   `tutor.js playScript · board.js playStep`

8. **Follow-ups.** Pressing the hotkey mid-answer cancels the model call, cuts the voice, lets the current drawing finish, snapshots the board, and listens again. The next answer builds on the same board.
   `tutor.js hotkey, interrupt`

The script format both halves agree on:

```
{ "topic": "Pythagorean theorem",
  "steps": [
    { "say": "Imagine a right triangle.",
      "actions": [ { "type": "draw_shape", "id": "t1", "shape": "triangle",
                     "points": [[100,380],[100,140],[340,380]] } ] },
    { "say": "The rule: a squared plus b squared equals c squared.",
      "actions": [ { "type": "write_formula", "latex": "a^2 + b^2 = c^2", "region": "right" } ] }
  ] }
```

Allowed actions: `draw_line`, `arrow`, `draw_shape`, `write_text`, `write_formula`, `label`, `highlight`, `erase`, `draw_svg`, and the board-laid-out helpers `array`, `linked_list`, `binary_tree`, `call_stack`, `flowchart` and `plot`.

05 Key logic highlights

`shell/llm/callModel.js`: the model chain

Everything that talks to a model goes through **callModel()**, so models can be swapped without touching the rest. It builds the chain from whichever keys exist (Groq first, then Gemini) and applies three time limits: **8 s** per whole-answer attempt, a per-model first-step limit when streaming (**3 s** for qwen, **6 s** for gpt-oss), and a **20 s** deadline for the whole call. Busy models (429/5xx) get one retry after a second unless Groq says to wait longer.

Streaming runs *without* JSON mode: Groq holds JSON-mode streams back until the whole answer is checked, which made streaming pointless. Instead, each step is validated as it arrives. If a stream breaks halfway, the steps already delivered stand. If qwen answers a clarifying question in its own shape (`{"question": ...}`), it is rescued as a one-step answer. If no stream yields a step, the whole-answer JSON-mode chain runs with the time left. qwen also gets `max_tokens: 950`, because Groq's free tier caps it at 1000 output tokens per minute and refuses any request that might exceed that.

`shell/tutor.js`: the conversation state machine

The tutor moves between `idle → listening → thinking → playing → idle`, and the hotkey means something different in each: stop the mic while listening; interrupt and listen again while thinking or playing. A **turn** counter drops stale work: every new question, interrupt or Esc bumps it, and any async result that comes back with an old turn number is ignored. An

`AbortController` per answer cancels the model call on interrupt, which also saves qwen's per-minute budget.

Steps arrive through a small async queue, so step 1 plays while the model is still writing step 4. The board has no cancel, so the tutor feeds it one step at a time and waits for an interrupted step to finish drawing before reading the board state for a follow-up. 18 of the 41 shell tests replay exact interleavings of these events.

`shell/llm/tidy.js`: fixing the model's layout mistakes

Models are bad at coordinates. `tidy.js` runs deterministic fixes on every step before it reaches the board, and never changes what is said or drawn, only where and how:

- labels touching a line are pushed to the nearest clear spot; labels pointing at shapes that were never drawn are re-pointed at the nearest real one;
- no note keeps an exact position: next to a shape it becomes that shape's label, otherwise it is placed by region;
- in follow-ups, new drawings that land on old notes, or inside an old shape, move as a group to free space found by `space.js`;
- pure maths in a note becomes a KaTeX formula, and maths in a spoken line becomes words (`(1/2)MR^2` → "one half MR squared");
- a plot or flowchart gets a cleared board when something is already drawn.

On the board side, `placeContent()` in `board/board.js` handles overflow: when a note fits nowhere it removes the oldest notes one at a time, never shapes or their labels. Tested with a crowded board and four note-heavy follow-ups: before this change all four wiped the diagram, after it none did.

06 Setup and execution

Keys go in a `.env` at the repo root, which git ignores:

```
GROQ_API_KEY=...      # console.groq.com, tried first
GEMINI_API_KEY=...    # aistudio.google.com, last fallback
```

One-time setup and run:

```
cd shell
npm install
python3 -m venv .venv
.venv/bin/pip install piper-tts==1.8.0
```

```
mkdir -p voices   # then download en_US-lessac-medium.onnx and .onnx.json (see shell/README.md)
npm start         # Ctrl/Cmd+Shift+Space to ask
```

Tests and tools:

```
node --test shell/tests/            # 41 tests
node --test board/tests/*.test.cjs  # 48 tests
node shell/tools/llm-test.js --stream               # model latency and schema check
npx electron shell/tools/board-snapshot.js script.json   # draw a script on the real board, save a PNG
```

> On Node 26, Electron's installer can exit "successfully" without unpacking the binary. `shell/README.md` has the one-line manual fix.

In the overlay: the top bar has **Pause/Resume**, **Stop**, **Type** (type a question or an answer instead of speaking) and **History** (saved boards: click one to redraw it and keep asking follow-ups on it). Drag the bar to move the window and drag its edges to resize. Esc closes it.

Written from the code in the WhiteBoardAI repository (Kabir: shell, Aarav: board). Timings are from the team's own runs on a ThinkPad T490.